

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 09 · LECTURE NOTES

Learning Without Interaction

Offline reinforcement learning and imitation.

MODULE OUTCOMES

- Explain behaviour cloning and compounding error.
- Explain DAgger and why it helps.
- Explain why naive off-policy learning fails offline.
- Explain conservative methods and what they constrain.
- Choose between imitation and offline reinforcement learning.

1 Imitation learning

BEHAVIOUR CLONING IS SUPERVISED LEARNING, AND ITS FAILURE IS SEQUENTIAL

Collect state-action pairs from an expert and fit a classifier or regressor from states to actions. That is the entire method — **it is CSCE 633, with states as inputs and expert actions as labels**, and every technique from that course applies unchanged.

And it frequently works, which is worth saying first because the literature emphasises its failure: **for many tasks behaviour cloning is the strongest option per unit of engineering effort**, and it is what most deployed imitation systems actually use.

The failure is compounding error. A small action error takes the agent slightly off the expert's state distribution, where the policy was trained on less data and is therefore less accurate, which produces a larger error, which takes it further off.

So the error grows quadratically in the horizon rather than linearly — and it is a distribution shift the agent causes itself, which is the part supervised learning has no mechanism to anticipate: the training distribution is fixed and the test distribution depends on the trained policy. **CSCE 753 Module 12's distribution shift, generated by the model rather than by the world.**

THE PROBLEM: the training data comes from the EXPERT's state distribution; the agent is evaluated on ITS OWN.

Dagger (dataset aggregation):

```
repeat:
    run the CURRENT POLICY to collect states
    ask the EXPERT what it would do in those states
    add those newly labelled states to the dataset
    retrain on the aggregated dataset
```

Now the training distribution converges onto the agent's own distribution, and the error becomes LINEAR in the horizon rather than quadratic.

THE COST: the expert must be queryable ONLINE, in states the expert never chose to visit. For a human expert this is laborious and sometimes impossible -- "what would you do here?" asked about a situation the person would never have got themselves into, and about which they may have no useful intuition.

WHICH IS WHY DAGGER IS COMMON IN SIMULATION, where the "expert" is a planner or a policy with privileged information that can be queried freely and cheaply, and RARE with human experts.

THIS PATTERN -- train on the distribution you will be evaluated on -- IS THE SAME LESSON AS CSCE 753 MODULE 12, arrived at from a different direction.

2 Offline reinforcement learning

NAIVE OFF-POLICY LEARNING FAILS OFFLINE, FOR ONE IDENTIFIABLE REASON

Q-learning is off-policy (Module 03 §2), so running it on a fixed logged dataset looks like it ought to work — the algorithm never required the data to come from its own policy. **It does not work**, and the reason is specific rather than general.

The target contains $\max_a Q(s',a)$, and the max may select an action that the dataset never contains at that state — so $Q(s',a)$ there is a pure extrapolation of the network, with no data behind it whatsoever.

Extrapolated values are essentially arbitrary and tend to be overestimates (Module 03 §4's maximisation bias, now operating on extrapolations rather than on noise), and the max actively seeks exactly those overestimates.

And with no interaction, nothing corrects it. Online, the agent would try the over-valued action, receive a poor reward, and learn; offline, the error propagates through bootstrapping indefinitely and the Q-values diverge. Which is why offline is strictly harder than off-policy, and why it needed its own methods rather than being a straightforward application of existing ones.

Method	Approach	What it constrains
BCQ / BEAR	Restrict the policy's actions to those close to actions present in the data.	The policy's action distribution — which works and requires a notion of 'close' in action space.
CQL (conservative Q-learning)	Add a penalty that pushes down Q on out-of-distribution actions.	The value function, which is cleaner — the policy is then free to be greedy, because the values it would exploit have already been suppressed.
IQL (implicit Q-learning)	Use expectile regression on the value function and never query Q at an unseen action at all.	It avoids the max entirely, so §2's failure mode cannot occur. Simple, strong, and few hyperparameters.
Decision transformer	Treat the trajectory as a sequence and train a model to predict actions conditioned on a desired return.	Nothing — it is supervised sequence learning (CSCE 638), which sidesteps the whole problem and gives up the ability to improve on the data by value propagation.

IQL is the one to try first. It never evaluates an action outside the dataset, so §2's failure mode cannot arise by construction rather than being penalised after the fact — and it has fewer hyperparameters than CQL, which matters in a setting where you cannot evaluate your hyperparameter choices online (§4).

3 Choosing between them

- **Expert data, and expert behaviour is what you want: behaviour cloning. Simpler, better understood, and frequently just as good** — and if the horizon is short or the task is forgiving, the compounding error of §1 may never bite.
- **Expert data, and the expert is queryable online: DAgger.** It addresses the compounding directly and cheaply, when the query is available.
- **Mixed-quality data, and you want to exceed the average demonstrator: offline reinforcement learning.** This is the case where it clearly wins — it can stitch good segments from several mediocre trajectories into a policy better than any single demonstration, which imitation cannot do by construction.
- **Narrow, uniformly expert data: offline reinforcement learning gains little**, because there is nothing better in the data to find, and the conservative constraints will hold it close to the demonstrations anyway.
- **And no reward labels at all: imitation, or inverse reinforcement learning to infer a reward function from the demonstrations** and then optimise it — which connects directly to Module 10 §4's preference learning.
- **'Can it beat the data?' is the question that separates the two families, and the honest answer depends on whether the data contains better behaviour somewhere to be stitched together.** If every trajectory is equally good, there is nothing to stitch.

4 Evaluating offline, which is the hard part

YOU CANNOT EVALUATE AN OFFLINE POLICY OFFLINE

To know a policy's value you have to run it, and in a genuinely offline setting you cannot — which means the entire model-selection and hyperparameter-tuning apparatus of CSCE 633 is unavailable.

Off-policy evaluation estimates the value from logged data — importance sampling, doubly robust estimators, fitted Q-evaluation — **and all of them have high variance when the evaluation policy differs substantially from the logging policy** (Module 05 §3's importance-weight explosion, in an evaluation role).

Which is exactly the situation in which you need the estimate: if the learned policy closely resembles the logging policy, there was little point learning it, and if it does not, the estimator is unreliable. The problem is structural, not a gap in the methods.

So the honest position is that offline training with a small, careful online evaluation is the realistic workflow — even a few hundred carefully monitored online episodes transform what you can claim — **and that purely offline claims should be stated as estimates with their variance**, which is this program's standard discipline applied to the one setting where it is hardest to meet.

WHERE THIS MATTERS MOST

Healthcare, education, and anything where exploration harms someone. You cannot try a random treatment to see what happens, which rules out every method in Modules 02 through 08 and makes this module the only applicable one.

Industrial control and robotics, where a bad action damages expensive equipment, and where years of logged operational data already exist from human or classical controllers.

Recommendation and dialogue systems, where enormous interaction logs exist and online experimentation is constrained by cost, risk, or policy.

And language model alignment. RLHF is offline reward modelling from human preference comparisons plus online policy optimisation against the learned reward (Module 10 §4) — **which is by far the largest deployed application of anything in this course**, and the reason the material in Module 10 deserves the attention it gets there rather than being a footnote.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Levine et al. — Offline Reinforcement Learning: Tutorial, Review, and Perspectives (free)	The reference for §2 and §4. The extrapolation analysis is the clearest statement of the core failure.
Ross, Gordon & Bagnell — A Reduction of Imitation Learning and Structured Prediction (DAgger) (free)	§1's method , with the quadratic-to-linear bound proved.
Kostrikov, Nair & Levine — Offline RL with Implicit Q-Learning (free)	§2's recommended method , and notably simple for what it achieves.
Fu et al. — D4RL: Datasets for Deep Data-Driven Reinforcement Learning (free)	Standard offline datasets , including the mixed-quality ones that make §3's stitching question concrete.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Collect expert data** from a trained policy and fit behaviour cloning. Report the performance gap.
- 2 **Measure how far the cloned policy's state distribution drifts** from the expert's over an episode.
- 3 **Plot error against horizon** and confirm it is superlinear.

- 4 **Implement DAgger** with the trained policy as the queryable expert, and confirm the improvement.
- 5 **Run plain Q-learning on a fixed offline dataset** and plot the Q-values. Confirm they diverge.
- 6 **Identify which actions the max is selecting** and confirm they are absent from the data.
- 7 **Implement a conservative penalty** and report the effect on both Q-values and performance.
- 8 **Implement IQL** and compare against CQL on a D4RL dataset.
- 9 **Build a mixed-quality dataset** and show offline RL beating the average demonstrator while behaviour cloning does not.
- 10 **Try off-policy evaluation** and compare its estimate against the true online return. Report the error.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is behaviour cloning, and what is its specific failure?
- 2 Why does the error grow quadratically?
- 3 What does DAgger change, and what does it require?
- 4 Why is DAgger common in simulation and rare with human experts?
- 5 Explain precisely why naive Q-learning fails offline.
- 6 Why does nothing correct the error?
- 7 Name four offline methods and what each constrains.
- 8 Which should you try first, and why?
- 9 What question separates imitation from offline RL?
- 10 Why can you not evaluate an offline policy offline?